

Student Name	CHOWDAM ESWAR	
Student Registration Number	252U1R6052	Class & Section: 203 AIML – 2A
Study Level : UG/PG	UG	Year & Term: II & I
Subject Name	Advanced Database Management Systems	

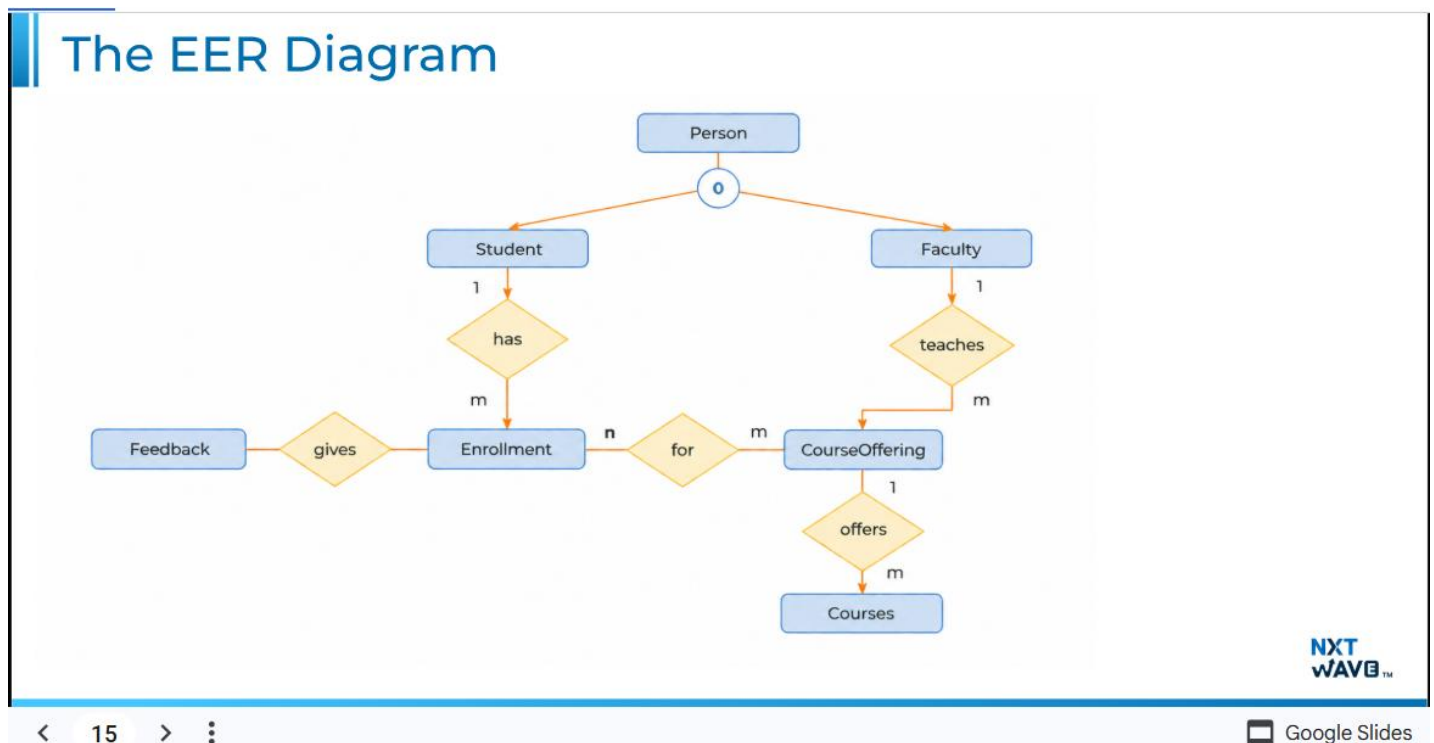
Name of the Assessment	Assignment – 1
Date of Submission	18 – 09 – 2026

Objective

This assignment presents the Enhanced Entity-Relationship (EER) design and the corresponding relational database schema for two independent systems a University System and a Food Delivery System. For each system, the EER diagram is given first, followed by the SQL Data Definition Language (DDL) statements that implement the diagram as a relational schema, and finally the mapping rules that justify how each EER construct (entity, specialization, 1:N relationship, and M:N relationship) was translated into tables, foreign keys, and check constraints.

Part A — University System

1. Enhanced ER (EER) Diagram



2. Relational Schema (SQL DDL)

The EER diagram above is implemented using the following seven tables.

```
-- =====
-- UNIVERSITY SYSTEM
-- =====

-- 1. PERSON
```

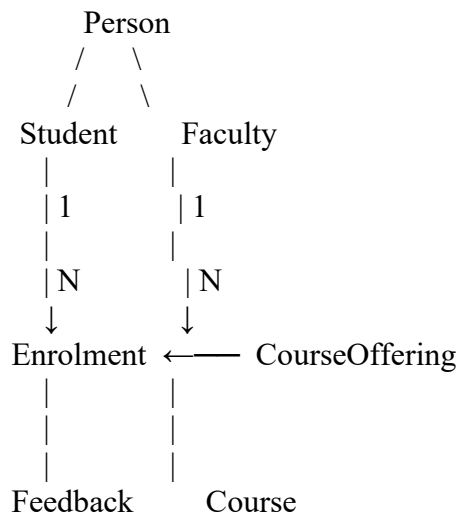
```
CREATE TABLE Person (  
    person_id INT PRIMARY KEY,  
    name VARCHAR(60) NOT NULL,  
    email VARCHAR(80) UNIQUE,  
    phone VARCHAR(15)  
);  
  
-- 2. STUDENT  
-- Person -> Student specialization  
CREATE TABLE Student (  
    person_id INT PRIMARY KEY,  
    roll_no VARCHAR(12) UNIQUE NOT NULL,  
    programme VARCHAR(10),  
    cgpa DECIMAL(3,2),  
    FOREIGN KEY (person_id)  
        REFERENCES Person(person_id)  
);  
  
-- 3. FACULTY  
-- Person -> Faculty specialization  
CREATE TABLE Faculty (  
    person_id INT PRIMARY KEY,  
    emp_code VARCHAR(12) UNIQUE NOT NULL,  
    department VARCHAR(40),  
    designation VARCHAR(30),  
    FOREIGN KEY (person_id)  
        REFERENCES Person(person_id)  
);  
  
-- 4. COURSE  
CREATE TABLE Course (  
    course_code VARCHAR(8) PRIMARY KEY,  
    title VARCHAR(120) NOT NULL,  
    credits INT CHECK (credits > 0)  
);  
  
-- 5. COURSE OFFERING  
-- Faculty 1 : N CourseOffering  
-- Course 1 : N CourseOffering  
CREATE TABLE CourseOffering (  
    offering_id INT PRIMARY KEY,
```

```
course_code VARCHAR(8) NOT NULL,
faculty_id INT NOT NULL,
semester VARCHAR(8),
year INT,
FOREIGN KEY (course_code)
REFERENCES Course(course_code),
FOREIGN KEY (faculty_id)
REFERENCES Faculty(person_id)
);

-- 6. ENROLMENT
-- Student M : N CourseOffering
-- Enrolment is the associative entity
CREATE TABLE Enrolment (
    student_id INT,
    offering_id INT,
    grade CHAR(2),
    PRIMARY KEY (student_id, offering_id),
    FOREIGN KEY (student_id)
        REFERENCES Student(person_id),
    FOREIGN KEY (offering_id)
        REFERENCES CourseOffering(offering_id)
);

-- 7. FEEDBACK
-- Feedback is attached to a particular Enrolment
CREATE TABLE Feedback (
    feedback_id INT PRIMARY KEY,
    student_id INT NOT NULL,
    offering_id INT NOT NULL,
    rating INT CHECK (rating BETWEEN 1 AND 5),
    comment VARCHAR(200),
    FOREIGN KEY (student_id, offering_id)
        REFERENCES Enrolment(student_id, offering_id)
);
```

3. Relationship / Mapping Diagram



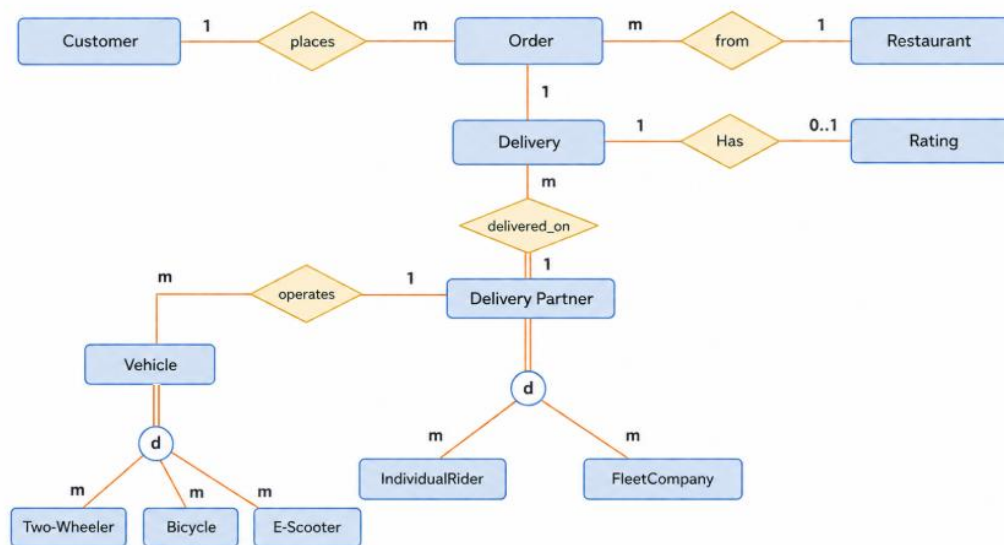
4. EER → Relational Mapping Rules

- **Person** → **Student** = specialization
- **Person** → **Faculty** = specialization
- **Student** → **Enrolment** = 1:N
- **Faculty** → **CourseOffering** = 1:N (teaches)
- **Course** → **CourseOffering** = 1:N
- **Student** ↔ **CourseOffering** = M:N, represented by Enrolment
- **Feedback** → **Enrolment** = feedback is attached to the aggregated M:N relationship

Part B — Food Delivery System

1. Enhanced ER (EER) Diagram

The EER Diagram



NXT
WAVE™

< 27 > ⋮

Google Slides

2. Relational Schema (SQL DDL)

The EER diagram above is implemented using the following twelve tables.

```
-- =====
-- FOOD DELIVERY SYSTEM
-- =====

-- 1. CUSTOMER
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    name        VARCHAR(60),
    phone       VARCHAR(15)
);

-- 2. RESTAURANT
CREATE TABLE Restaurant (
    restaurant_id INT PRIMARY KEY,
    name          VARCHAR(80),
```

```
area      VARCHAR(40)
);

-- 3. PARTNER
-- 'R' = Individual Rider
-- 'F' = Fleet Company
CREATE TABLE Partner (
    partner_id INT PRIMARY KEY,
    partner_type CHAR(1),
    CHECK (partner_type IN ('R', 'F'))
);

-- 4. INDIVIDUAL RIDER
CREATE TABLE IndividualRider (
    partner_id INT PRIMARY KEY
    REFERENCES Partner(partner_id),
    rider_id INT UNIQUE,
    name VARCHAR(60),
    license_no VARCHAR(20)
);

-- 5. FLEET COMPANY
CREATE TABLE FleetCompany (
    partner_id INT PRIMARY KEY
    REFERENCES Partner(partner_id),
    legal_name VARCHAR(120),
    gst_number VARCHAR(15)
);

-- 6. ORDER
-- Customer 1 : N Order
-- Restaurant 1 : N Order
-- ORDER is a reserved keyword in SQL, so we use Orders.
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT REFERENCES Customer(customer_id),
    restaurant_id INT REFERENCES Restaurant(restaurant_id),
    total_amount DECIMAL(10,2),
    paid_via VARCHAR(10)
);
```

-- 7. VEHICLE

-- Partner 1 : N Vehicle

-- FK is placed on the N side.

```
CREATE TABLE Vehicle (  
    vehicle_id INT PRIMARY KEY,  
    registration VARCHAR(12),  
    energy_note VARCHAR(40),  
    partner_id INT REFERENCES Partner(partner_id)  
);
```

-- 8. TWO-WHEELER

-- Vehicle specialization

```
CREATE TABLE TwoWheeler (  
    vehicle_id INT PRIMARY KEY REFERENCES Vehicle(vehicle_id),  
    engine_cc INT  
);
```

-- 9. BICYCLE

-- Vehicle specialization

```
CREATE TABLE Bicycle (  
    vehicle_id INT PRIMARY KEY REFERENCES Vehicle(vehicle_id),  
    gear_count INT  
);
```

-- 10. E-SCOOTER

-- Vehicle specialization

```
CREATE TABLE EScooter (  
    vehicle_id INT PRIMARY KEY REFERENCES Vehicle(vehicle_id),  
    battery_wh INT  
);
```

-- 11. DELIVERY

-- One row per order

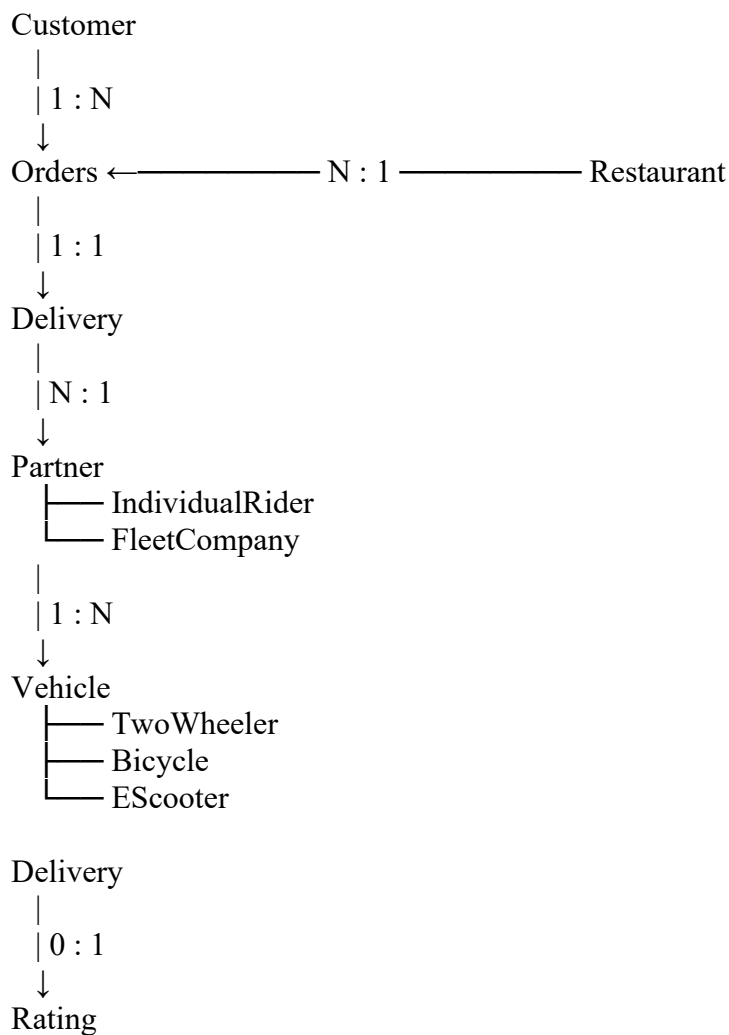
-- Order 1 : 1 Delivery

-- Partner 1 : N Delivery

```
CREATE TABLE Delivery (  
    order_id INT PRIMARY KEY REFERENCES Orders(order_id),  
    partner_id INT REFERENCES Partner(partner_id),  
    delivered_on TIMESTAMP  
);
```

```
-- 12. RATING
-- One Delivery can have 0 or 1 Rating
-- UNIQUE(order_id) enforces this.
CREATE TABLE Rating (
    rating_id INT PRIMARY KEY,
    order_id INT UNIQUE REFERENCES Delivery(order_id),
    stars INT CHECK (stars BETWEEN 1 AND 5),
    comment VARCHAR(200)
);
```

3. Relationship / Mapping Diagram



4. EER → Relational Mapping Rules

- **Partner** → **IndividualRider** = specialization
- **Partner** → **FleetCompany** = specialization
- **Customer** → **Order** = 1:N (places)
- **Restaurant** → **Order** = 1:N (from)
- **Order** → **Delivery** = 1:1 (has / one delivery per order)
- **Delivery** → **Partner** = N:1 (delivered_by)
- **Partner** → **Vehicle** = 1:N (operates)
- **Vehicle** → **TwoWheeler** = specialization
- **Vehicle** → **Bicycle** = specialization
- **Vehicle** → **EScooter** = specialization
- **Delivery** → **Rating** = 0:1 (has) — a delivery may have zero or one rating

Conclusion

Both schemas follow consistent design principles: every specialization (Person→Student/Faculty, Partner→IndividualRider/FleetCompany, Vehicle→TwoWheeler/Bicycle/EScooter) is implemented using shared primary keys with the parent table, every 1:N relationship places the foreign key on the ‘many’ side, and every M:N relationship (Student–CourseOffering) is resolved using an associative entity (Enrolment) with a composite primary key. CHECK constraints (e.g., on rating, stars, credits, and partner_type) enforce domain integrity directly at the schema level, keeping both databases in at least Third Normal Form (3NF).